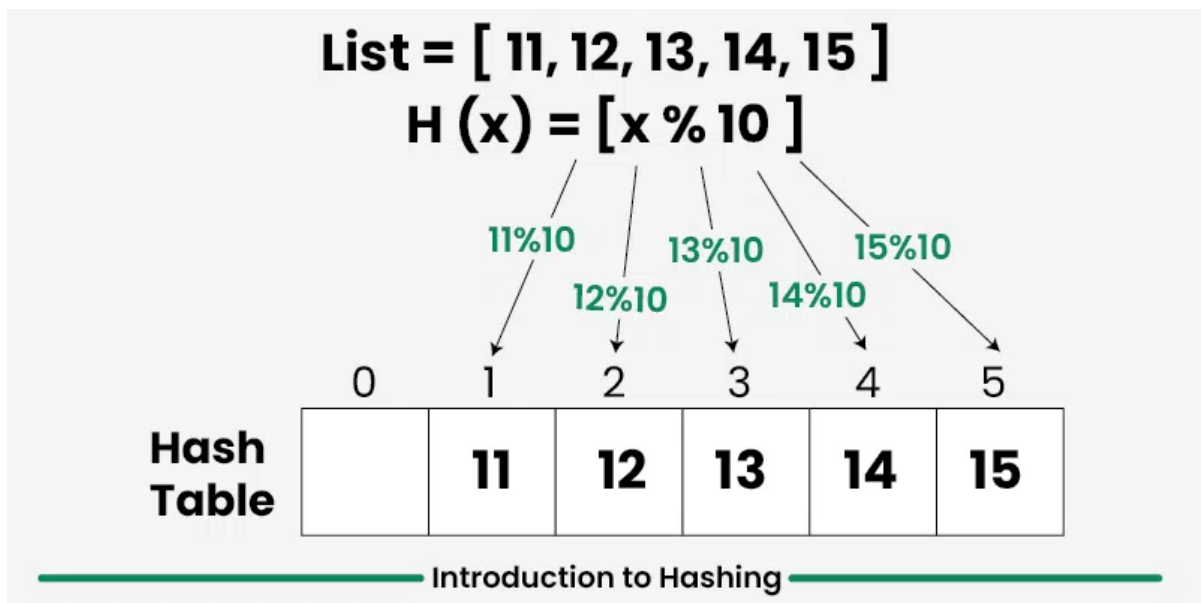


Introduction to Hashing

Last Updated : 21 Mar, 2025



Hashing refers to the process of generating a small sized output (that can be used as index in a table) from an input of typically large and variable size. Hashing uses mathematical formulas known as hash functions to do the transformation. This technique determines an index or location for the storage of an item in a data structure called Hash Table.



Introduction to Hashing

Hash Table Data Structure Overview

- It is one of the most widely used data structure after arrays.
- It mainly supports search, insert and delete in $O(1)$ time on average which is more efficient than other popular data structures like arrays, Linked List and [Self Balancing BST](#).
- We use hashing for dictionaries, frequency counting, maintaining data for quick access by key, etc.
- Real World Applications include Database Indexing, Cryptography, Caches, Symbol Table and Dictionaries.
- There are mainly two forms of hash typically implemented in programming languages.

Hash Set : Collection of unique keys (Implemented as [Set in Python](#), [Set in JavaScript](#), [unordered_set in C++](#) and [HashSet in Java](#)).

Hash Map : Collection of key value pairs with keys being unique (Implemented as [dictionary in Python](#), [Map in JavaScript](#), [unordered_map in C++](#) and [HashMap in Java](#))

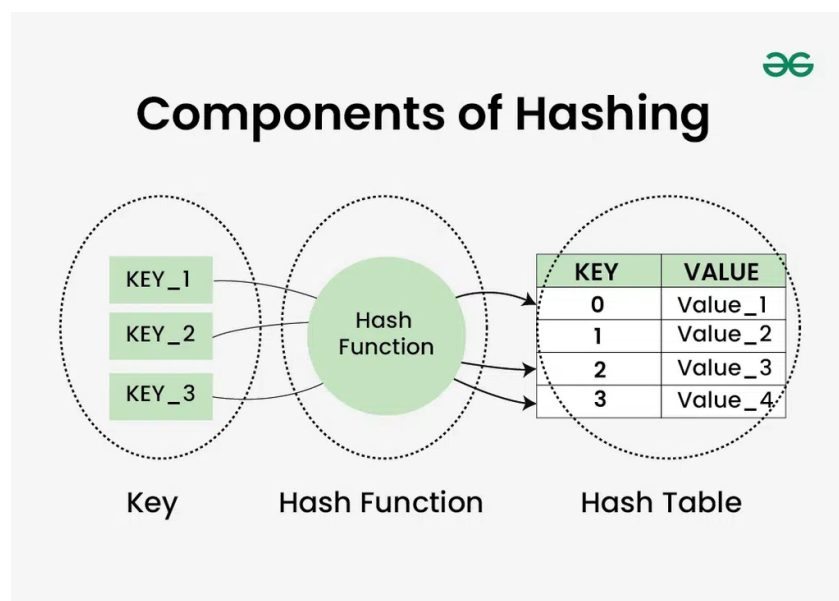
Situations Where Hash is not Used

- Need to maintain sorted data along with search, insert and delete. We use a self balancing BST in these cases.
- When Strings are keys and we need operations like prefix search along with search, insert and delete. We use Trie in these cases.
- When we need operations like floor and ceiling along with search, insert and/or delete. We use Self Balancing BST in these cases.

Components of Hashing

There are majorly three components of hashing:

1. **Key:** A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. **Hash Function:** Receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index .
3. **Hash Table:** Hash table is typically an array of lists. It stores values corresponding to the keys. Hash stores the data in an associative manner in an array where each data value has its own unique index.



How does Hashing work?

Suppose we have a set of strings {"ab", "cd", "efg"} and we would like to store it in a table.

- **Step 1:** We know that hash functions (which is some mathematical formula) are used to calculate the hash value which acts as the index of the data structure where the value will be stored.
- **Step 2:** So, let's assign
 - "a" = 1,
 - "b"=2, .. etc, to all alphabetical characters.
- **Step 3:** Therefore, the numerical value by summation of all characters of the string:

- $"ab" = 1 + 2 = 3,$
- $"cd" = 3 + 4 = 7,$
- $"efg" = 5 + 6 + 7 = 18$

- **Step 4:** Now, assume that we have a table of size 7 to store these strings. The hash function that is used here is the sum of the characters in **key mod Table size** . We can compute the location of the string in the array by taking the **sum(string) mod 7** .
- **Step 5:** So we will then store
 - "ab" in $3 \bmod 7 = 3,$
 - "cd" in $7 \bmod 7 = 0,$ and
 - "efg" in $18 \bmod 7 = 4.$

Mapping Key with indices of Array

0	1	2	3	4	5	6
cd			ab	egf		

The above technique enables us to calculate the location of a given string by using a simple hash function and rapidly find the value that is stored in that location. Therefore the idea of hashing seems like a great way to store (key, value) pairs of the data in a table.

What is a Hash function?

A [hash function](#) creates a mapping from an input key to an index in hash table, this is done through the use of mathematical formulas known as hash functions. For example: Consider phone numbers as keys and a hash table of size 100. A simple example hash function can be to consider the last two digits of phone numbers so that we have valid array indexes as output. A good hash function should have the following properties:

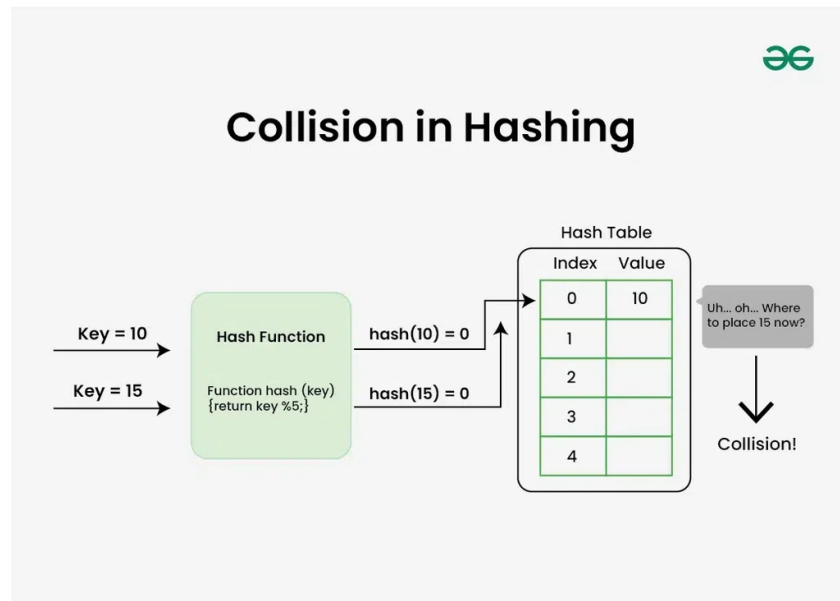
1. Efficient
2. Should uniformly distribute the keys to each index of hash table.
3. Should minimize collisions (This and the below are mainly derived from the above 2nd point)
4. Should have a low load factor (number of items in the table divided by the size of the table).

What is Collision in Hashing?

When two or more keys have the same hash value, a **collision** happens. If we consider the above example, the hash function we used is the sum of the letters, but if we examined the hash function closely then the problem can be easily

visualised that for different strings same hash value is being generated by the hash function.

For example: {"ab", "ba"} both have the same hash value, and string {"cd","be"} also generate the same hash value, etc. This is known as **collision** and it creates problem in searching, insertion, deletion, and updating of value.



Collision in Hashing

The probability of a hash collision depends on the size of the algorithm, the distribution of hash values and the efficiency of Hash function. To handle this collision, we use [Collision Resolution Techniques](#).

What is meant by Load Factor in Hashing?

The [load factor](#) of the hash table can be defined as the number of items the hash table contains divided by the size of the hash table. Load factor is the decisive parameter that is used when we want to rehash the previous hash function or want to add more elements to the existing hash table.

It helps us in determining the efficiency of the hash function i.e. it tells whether the hash function which we are using is distributing the keys uniformly or not in the hash table.

$$\text{Load Factor} = \text{Total elements in hash table} / \text{Size of hash table}$$

What is Rehashing?

As the name suggests, [rehashing](#) means hashing again. Basically, when the load factor increases to more than its predefined value (the default value of the load factor is 0.75), the complexity increases. So to overcome this, the size of the array is increased (doubled) and all the values are hashed again and stored in the new double-sized array to maintain a low load factor and low complexity.

How to Create Your Own Hash Table?

- [You Own Hash Table with Chaining](#)
- [Your Own Hash Table with Linear Probing in Open Addressing](#)
- [Your Own Hash Table with Quadratic Probing in Open Addressing](#)

 Comment

More info 

Advertise with us

Next Article 

What is Hashing?

Similar Reads

Hashing in Data Structure

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. Hashing involves mapping data to a specific index in a hash table (an array of...

 3 min read

Introduction to Hashing

Hashing refers to the process of generating a small sized output (that can be used as index in a table) from an input of typically large and variable size. Hashing uses mathematical formulas known as hash...

 7 min read

What is Hashing?

Hashing refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an index or location for the...

 3 min read

Index Mapping (or Trivial Hashing) with negatives allowed

Index Mapping (also known as Trivial Hashing) is a simple form of hashing where the data is directly mapped to an index in a hash table. The hash function used in this method is typically the identity...

 7 min read

Separate Chaining Collision Handling Technique in Hashing

Separate Chaining is a collision handling technique. Separate chaining is one of the most popular and commonly used techniques in order to handle collisions. In this article, we will discuss about what is...

🕒 3 min read

Open Addressing Collision Handling technique in Hashing

Open Addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number...

🕒 7 min read

Double Hashing

Double hashing is a collision resolution technique used in hash tables. It works by using two hash functions to compute two different hash values for a given key. The first hash function is used to comput...

🕒 15+ min read

Load Factor and Rehashing

Prerequisites: Hashing Introduction and Collision handling by separate chaining How hashing works: For insertion of a key(K) - value(V) pair into a hash map, 2 steps are required: K is converted into a small...

🕒 15+ min read

Easy problems on Hashing



Intermediate problems on Hashing

